FAKE REVIEW DETECTION ON

E-COMMERCE PRODUCT

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

AWARD OF THE DEGREE OF

MASTER OF SCIENCE

IN

APPLIED DATA SCIENCE

BY

DINAKARRAJ M (REG NO. RA2432014010123)

LOKJITH D (REG NO. RA2432014010124)

SANJAY J (REG NO. RA2432014010143)

JAWAHAR S (REG NO. RA2432014010160)

UNDER THE GUIDANCE OF

Dr. Brindha S M.Sc., M.Phil., Ph. D.

DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Kattankulathur – 603 203

Chennai, Tamilnadu

OCTOBER - 2025

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**FAKE REVIEW DETECTION ON E-COMMERCE PRODUCT**" is a Bonafide work carried out by **DINAKARRAJ M** (RA2432014010123), **LOKJITH D** (RA2432014010124), **SANJAY J** (RA2432014010143), **JAWAHAR S** (RA2432014010160) under my supervision for the award of the Degree of Master of Science in Applied Data Science. To my knowledge the work reported herein is the original work done by these students.

Dr. Brindha S

Assistant Professor,

Department of Computer Applications

(GUIDE)

Dr.P.J.Arul Leena Rose

Professor & Head,

Department of Computer Applications

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank the people who helped me to complete this project.

We take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

We are thankful to **Dr.T.R.Paarivendhar**, Chancellor, and **Prof.A.Vinay Kumar**, Pro Vice-Chancellor (SBL), SRM Institute of Science & Technology who gave us the platform to establish me to reach greater heights.

We earnestly thank **Dr.A.Duraisamy**, Dean, Faculty of Science and Humanities, SRM Institute of Science & Technology who always encourage us to do novel things.

A great note of gratitude to **Dr. S. Albert Antony Raj**, Deputy Dean, Faculty of Science and Humanities for his valuable guidance and constant Support to do this Project.

We express our sincere thanks to **Dr. P.J.Arul Leena Rose**, Professor & Head for her support to execute all incline in learning.

It is our delight to thank our project guide **Dr. Brindha S,** Assistant Professor, Department of Computer Applications for his help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

We convey our gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during the reviews.

Our gratitude to friends and people who are known and unknown to me who helped in carrying out this project work a successful one.

DINAKARRAJ M

LOKJITH D

SANJAY J

JAWAHAR S

TABLE OF CONTENTS

ABSTRACT

The increasing reliance on online reviews in e-commerce necessitates robust methods to identify and filter fake or deceptive reviews. These reviews, often biased or fabricated, can mislead consumers and distort market perceptions. This project, "**Fake Review Detection On E-Commerce Product**", presents a machine learning-based system designed to automatically classify the authenticity of product reviews. The system analyses the textual content of reviews using Natural Language Processing (NLP) techniques and employs supervised learning algorithms for classification. The project workflow commenced with data acquisition from Excel files containing e-commerce reviews and associated metadata. Preprocessing involved text cleaning operations such as lowercasing, removal of special characters and URLs, and whitespace normalization, implemented within a Google Colab environment. Feature engineering primarily utilized the TF-IDF (Term Frequency-Inverse Document Frequency) vectorization method to convert cleaned text into numerical representations suitable for machine learning models. N-grams (unigrams and bigrams) were included to capture some contextual information. Several classification models were explored, including Logistic Regression, Random Forest, XGBoost, Naive Bayes, and Support Vector Machines (SVM). Due to the absence of ground truth labels in the dataset, a heuristic approach was adopted for demonstration, labelling reviews with very short lengths (less than 5 words) and a maximum rating (5 stars) as potentially "Fake". Models were trained and evaluated using standard metrics like accuracy, precision, recall, F1-score, and ROC-AUC. Notably, XGBoost and Random Forest showed promising results on the heuristically labelled test data within the notebook. For deployment, the trained TF-IDF vectorizer and five classifiers were saved using joblib. A web application was developed using the Flask framework, providing a user-friendly interface (index.html styled with style.css). Users can input review text, choose a classification model, and receive a prediction ("Fake Review" or "Genuine Review"). This application serves as a proof-of-concept, demonstrating the feasibility of using ML for automated fake review detection, which can contribute to enhancing trust and transparency in online marketplaces.

LIST OF TABLES

LIST OF FIGURES

1. INTRODUCTION

The digital transformation of commerce has precipitated a significant shift in consumer behaviour, with online product reviews emerging as a critical determinant in purchasing decisions. This form of user-generated content functions as essential social proof, offering prospective buyers valuable peer insights. However, this very influence has created substantial incentives for review manipulation, leading to the proliferation of fake reviews designed to artificially inflate or deflate product reputations. This phenomenon introduces significant noise into the information ecosystem, undermining consumer trust, distorting market dynamics, and necessitating robust detection methodologies. Conventional approaches, relying predominantly on manual moderation or simplistic rule-based filters, face severe limitations in scalability, accuracy, and the ability to counteract sophisticated deception tactics employed by malicious actors. These challenges underscore the urgent need for advanced, automated systems capable of nuanced content analysis. This project, titled Fake Review Detection On E-commerce Product, directly confronts these issues by developing and implementing a machine learning-based system engineered to classify review authenticity based on intrinsic linguistic characteristics derived through Natural Language Processing (NLP).

1.1 Background of the Study

The architecture of an effective fake review detection system integrates several key technological components: data preprocessing pipelines, feature engineering methodologies, machine learning classification algorithms, and often a deployment framework for practical application. The historical evolution of review analysis began with rudimentary keyword spotting and metadata heuristics (e.g., review velocity, rating distribution). The advent of machine learning introduced more sophisticated techniques capable of learning complex patterns from data. Current state-of-the-art approaches often leverage deep learning models; however, traditional machine learning classifiers combined with robust feature engineering remain highly effective and computationally more tractable for many applications.

The necessity for this project is driven by the specific need to provide an accessible, automated tool for evaluating review authenticity based purely on textual content. While metadata analysis offers valuable signals, linguistic patterns within the review text itself often contain subtle indicators of deception. Addressing this requires effective text representation and classification. The challenge of text representation is tackled here using the TF-IDF (Term Frequency-Inverse Document Frequency) vectorization technique. TF-IDF captures the relative importance of words within a review compared to their frequency across the entire dataset, providing a numerical feature space suitable for machine learning.

The classification task is addressed by training and evaluating a suite of standard supervised learning algorithms, including Logistic Regression, Random Forest, XGBoost, Naive Bayes, and Support Vector Machines (SVM). These models learn decision boundaries within the TF-IDF feature space to distinguish between classes. A significant constraint in this domain is often the lack of large, reliably labelled datasets. For this project, a heuristic labelling strategy (based on review length and rating) was employed for demonstration purposes, allowing for the development and evaluation of the end-to-end pipeline, albeit with the understanding that real-world performance would depend on ground-truth labels. Finally, the practical utility is demonstrated through deployment via a Flask web application, enabling user interaction and real-time prediction using selected trained models. By integrating these components, this study delivers a functional prototype for content-based fake review detection.

1.2 Statement of Objectives

The primary objective of this project is to develop, implement, and evaluate a prototype system for the automated detection of potentially fake e-commerce reviews using machine learning and NLP techniques applied to review text. The specific objectives defined to guide this process are:

1.2.1. Develop a Data Processing Pipeline: To implement procedures within a Google Colab environment for loading e-commerce review data from Excel files, cleaning the

raw text (lowercasing, removing URLs/special characters), and preparing it for feature extraction.

1.2.2. Implement Text Feature Engineering: To utilize the TF-IDF vectorization technique (including unigrams and bigrams) via Scikit-learn to transform the pre-processed review text into a high-dimensional numerical feature representation.

1.2.3. Apply Heuristic Labelling: To generate binary labels (Fake/Genuine) for the dataset based on predefined heuristics (review length and rating) to enable supervised model training in the absence of ground truth data.

1.2.4. Train and Evaluate Classification Models: To train multiple machine learning classifiers (Logistic Regression, Random Forest, XGBoost, Naive Bayes, SVM) on the engineered features and evaluate their performance using standard metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC) on a held-out test set.

1.2.5. Web Application Development: Developed a web application using the Flask framework. This application loads the pre-trained models (Logistic Regression, Random Forest, Naive Bayes, SVM, XGBoost) and their corresponding TF-IDF vectorizers, accepts user input review text via an HTML interface, performs prediction based on the selected model, and displays the result (Fake/Genuine).

1.3 Scope of the Project

The scope of the Fake E-commerce Review Detection project is defined by the specific components implemented and the functionalities offered within this prototype system.

The project is limited to the following:

- **Data Source:** Analysis is performed on a subset of the large-scale Amazon Product Reviews 2023 dataset, originally collected by McAuley Lab. This subset was obtained via GitHub (https://amazon-reviews-2023.github.io/) and is distributed under the MIT License, granting permission for free use, modification, and distribution with attribution. The data was loaded for this

project from local Excel files (Subscription_box.xlsx, subscription_box-meta.xlsx) derived from this source.

- **Feature Scope:** Detection relies exclusively on features derived from the review text content via TF-IDF. Metadata features (e.g., reviewer history, timestamps, verified purchase status, product details), beyond the rating used for heuristic labelling, are not used in the deployed models, although some were explored during notebook development.

- **Labelling Method:** Authenticity labels (Fake/Genuine) are heuristically generated based on text length and rating for training/evaluation purposes. The system is not trained or evaluated on a dataset with verified ground truth labels.

- **Deployed Models:** The Flask web application deploys five classification models (Logistic Regression, Random Forest, Naive Bayes, SVM, XGBoost) that were trained and saved during the development phase.

- **Development & Deployment Environment:** Model development and training were performed using Google Colab. The web application is developed using Flask (in VS Code) with its built-in development server and presented via a basic HTML/CSS interface. It is **not** integrated into a live e-commerce platform or optimized for production-level traffic.

- **User Interaction:** The application handles single review predictions via manual text input through the web interface. Batch processing or API access is outside the current scope.

The project encompasses the following core functionalities:

- Loading and preprocessing of review text data.
- TF-IDF feature extraction using Scikit-learn.
- Training and evaluation of multiple classification models (within the notebook).
- Saving and loading of trained models and vectorizers using joblib.
- A Flask-based web backend for handling user requests and serving predictions.
- A basic HTML/CSS frontend allowing users to input text, select a model, and view the classification result.

2. LITERATURE SURVEY

This chapter reviews the theoretical background and established methodologies relevant to the Fake review detection on e-commerce product project. A comprehensive analysis of existing literature on Fake review detection, Natural Language Processing (NLP) techniques for text analysis, machine learning classification algorithms, and text feature engineering provides the necessary context for the approaches adopted in this study. Fake reviews pose a significant threat to the integrity of online marketplaces, influencing consumer behaviour and distorting product perception. This survey explores various computational techniques developed to automatically identify such deceptive content, forming the foundation for the system designed herein, which focuses on linguistic feature analysis and supervised classification. By understanding the established methods and their limitations, this project aims to implement and evaluate a practical system for detecting fake reviews based on their textual properties.

2.1 Natural Language Processing Fundamentals for Review Analysis

At the core of analysing review text lies Natural Language Processing (NLP), a field of computer science and artificial intelligence focused on enabling computers to understand, interpret, and manipulate human language. For fake review detection based on content, several fundamental NLP techniques are typically employed during the preprocessing stage to transform raw, unstructured text into a format suitable for analysis.

2.1.1. Text Cleaning and Normalization: Before feature extraction, raw review text must be cleaned and standardized. Common steps include:

- Lowercasing: Converting all text to lowercase ensures consistency (e.g., "Great" and "great" are treated as the same word).
- Removal of Noise: Eliminating irrelevant characters or elements like HTML tags, URLs, special characters, and excessive punctuation that do not typically contribute semantic value for classification. Regular expressions are commonly used for this task.
- Tokenization: Breaking down the text into individual words or "tokens".

- Stopword Removal: Filtering out common words (e.g., "the", "is", "a", "in") that occur frequently but often carry little discriminatory meaning. Libraries like NLTK provide standard stopword lists.

- Stemming/Lemmatization (Optional): Reducing words to their root form (e.g., "running" -> "run") to group related terms. While not explicitly used in the final feature extraction of this project, these are common NLP preprocessing steps.

2.1.2. Rationale: These preprocessing steps aim to reduce the dimensionality and noise in the text data, focusing the analysis on potentially more informative terms, which is crucial for the effectiveness of subsequent feature engineering and machine learning stages.

2.2 Text Representation:

From Text to Features Machine learning algorithms operate on numerical data, necessitating the conversion of pre-processed text into quantitative feature vectors. Several techniques exist for this transformation.

2.2.1. Bag-of-Words (BoW): A simple approach where each document (review) is represented as a vector indicating the presence or frequency of words from a predefined vocabulary, disregarding grammar and word order.

2.2.2. Term Frequency-Inverse Document Frequency (TF-IDF): This project primarily utilizes TF-IDF, a more sophisticated weighting scheme than basic BoW.

- Term Frequency (TF): Measures how often a term appears in a specific document.

- Inverse Document Frequency (IDF): Measures the importance of a term across the entire corpus. Common terms receive lower IDF scores, while rarer terms receive higher scores, highlighting potentially more distinctive words.

- TF-IDF Weight: Calculated as TF × IDF. This score reflects the importance of a term within a specific document relative to the corpus. Each review is thus represented as a vector of TF-IDF weights.

- N-grams: To capture some local word order information, TF-IDF can be extended to consider sequences of N adjacent words (n-grams). This project employs unigrams (N=1) and bigrams (N=2).

2.2.3. Word Embeddings (Advanced): Techniques like Word2Vec, GloVe, or those derived from transformer models (like BERT) represent words as dense vectors in a continuous space, capturing semantic relationships. While powerful, they often require more complex model architectures (like deep learning) and were not the primary focus of the deployed models in this project.

2.2.4. Choice Rationale: TF-IDF was chosen for its proven effectiveness in text classification tasks, its ability to handle sparse data, and its compatibility with traditional machine learning classifiers like those implemented (Logistic Regression, SVM, Random Forest, etc.). It provides a balance between representational power and computational efficiency suitable for this prototype.

2.3 Supervised Machine Learning for Classification

Once reviews are represented as numerical vectors, supervised machine learning algorithms can be trained to classify them as "Fake" or "Genuine", provided labelled training data exists.

2.3.1. Logistic Regression: A linear model that estimates the probability of a binary outcome (Fake/Genuine) using a logistic function. It's computationally efficient and relatively interpretable.

2.3.2. Naive Bayes (MultinomialNB): A probabilistic classifier based on Bayes' theorem, assuming independence between features (words/n-grams). Often performs well on text data despite the potentially unrealistic independence assumption.

2.3.3. Support Vector Machines (SVM): A classifier that finds an optimal hyperplane to separate classes in the high-dimensional feature space. Linear SVMs are particularly effective for text classification.

2.3.4. Ensemble Methods (Random Forest, XGBoost):

- **Random Forest:** Builds multiple decision trees on different subsets of data and features, aggregating their predictions (e.g., by majority vote) to improve robustness and reduce overfitting.

- **Gradient XGBoost (Extreme Boosting):** An efficient and scalable implementation of gradient boosting, which builds trees sequentially, each correcting the errors of the previous ones. Often achieves high performance.

2.3.5. Deep Learning (LSTM): Long Short-Term Memory networks, a type of RNN, can model sequential dependencies in text. While an LSTM structure was defined in the notebook, its implementation and evaluation appeared incomplete compared to the traditional classifiers.

2.3.6. Heuristic Labelling Impact: It is important to reiterate that the models in this project were trained using heuristically generated labels. The performance reflects their ability to learn this heuristic rather than true underlying patterns of deception present in a ground-truth dataset.

2.4 Feature Engineering Approaches in Review Analysis

Beyond basic text representation like TF-IDF, the literature explores various other features potentially indicative of fake reviews. While only TF-IDF was used in the deployed models, some simpler linguistic features were calculated during the notebook analysis:

- **Review Length & Character Count:** Extremely short or unusually long reviews can sometimes be suspicious.

- **Punctuation & Capitalization:** Overuse of exclamation marks or excessive capitalization might signal emotional exaggeration often found in fake reviews.

- **Sentiment Analysis:** While genuine reviews exhibit a range of sentiments, fake reviews might show unnaturally extreme positive or negative scores.

- **Readability Scores:** Metrics like Flesch-Kincaid could potentially differentiate writing styles.

- **Psycholinguistic Features:** Using tools like LIWC (Linguistic Inquiry and Word Count) to analyse pronoun usage, emotional tone, cognitive complexity, etc., has shown promise in deception detection studies.

- **N-gram Analysis:** Already incorporated via TF-IDF, but deeper analysis of specific informative n-grams is common.

Incorporating such features alongside TF-IDF, as was done for the XGBoost model in the notebook analysis, can potentially enhance model performance by providing different types of information.

2.5 Evaluation Methodologies for Detection Systems

Evaluating the performance of a fake review detection system is crucial. Standard classification metrics are typically used, especially considering that fake reviews often constitute the minority class (imbalanced dataset).

- **Accuracy:** Overall percentage of correctly classified reviews. Can be misleading in imbalanced datasets.

- **Precision:** Of the reviews classified as fake, what proportion are actually fake? (True Positives / (True Positives + False Positives)). Important for minimizing disruption from flagging genuine reviews.

- **Recall (Sensitivity):** Of all the actual fake reviews present, what proportion were correctly identified? (True Positives / (True Positives + False Negatives)). Important for catching as many fakes as possible.

- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric that balances both. Often a key metric for imbalanced classes.

- **ROC Curve & AUC:** The Receiver Operating Characteristic curve plots the True Positive Rate against the False Positive Rate at various thresholds. The Area Under the Curve (AUC) summarizes the model's ability to distinguish between classes across all thresholds. An AUC of 1.0 is perfect, while 0.5 is random guessing.

- **Confusion Matrix:** A table showing the counts of True Positives, True Negatives, False Positives, and False Negatives, providing a detailed breakdown of classification performance.

This project utilized Accuracy, Precision, Recall, F1-score (via classification_report), ROC-AUC (roc_auc_score), and visual tools like Confusion Matrices, ROC curves, bar charts, and radar charts for comprehensive model evaluation within the notebook environment.

3. SOFTWARE REQUIREMENT ANALYSIS

The development of the **FAKE REVIEW DETECTION ON E-COMMERCE PRODUCT** system necessitates a clear definition of the computational and environmental requirements to ensure successful implementation, robust operation, and straightforward deployment. This chapter details the technical prerequisites, providing specifications for the hardware, software, and the core features of the developed application. This analysis underpins the design and implementation choices, ensuring the final system is both technically feasible and operationally sound for its purpose as a prototype detection tool.

3.1 HARDWARE SPECIFICATION

The Fake review detection on e-commerce product system is implemented primarily in Python, involving both offline model training (potentially resource-intensive depending on data size) and a lightweight web application for prediction. The requirements reflect standard development and deployment needs for such a system.

3.1.1. Minimum Recommended Hardware This configuration is suitable for local development, model training on moderate datasets, testing, and single-user deployment of the Flask application.

.

Table 1 ***Table 3.1: Minimum Recommended Hardware***

Component	**Specification**
Processor (CPU)	Dual-core 2.0 GHz or equivalent
Random Access Memory (RAM)	8 GB

Storage	5 GB Free Disk Space
Network Interface	Stable Internet Connection (5 Mbps minimum)

3.1.2. Optimal Hardware Specification

This configuration is recommended for handling larger datasets during training, faster development iterations, and potentially supporting a small number of concurrent users if deployed internally.

Table 2 ***Table 3.2: Optimal Hardware Specification***

Component	**Specification**
Processor (CPU)	Quad-core 3.0 GHz or higher
Random Access Memory (RAM)	16 GB or higher
Storage	256 GB SSD (Solid State Drive)
Network Interface	High-Speed Fiber/Ethernet (50 Mbps minimum)

While model training benefits from better hardware, the prediction phase via the Flask application using pre-trained joblib models is generally lightweight and less demanding.

3.2 SOFTWARE SPECIFICATION

The project relies on standard open-source software components within the Python ecosystem, ensuring portability and ease of setup**.**

3.2.1. Operating System and Platform

The system is designed for cross-platform compatibility, leveraging the power of Python.

Table 3 ***Table 3.3: Operating System and Platform Requirements***

Operating System	Windows 10/11, macOS (Ventura or later), or Linux (Ubuntu 20.04+)
Virtualization	Python 3.x

3.2.2. **Core Programming and Library Stack**

The application relies on a modern set of libraries for its functionality. They are mentioned in the Table 3.4.

Table 4 ***Table 3.4: Core Programming Stack***

Component	Role in the System
Python 3.10 or later	The core execution language for all application logic.
Flask	Micro web framework used for building the backend API and serving the user interface.

Scikit-learn	Primary machine learning library for TF-IDF vectorization, classifiers (LogReg, RF, NB, SVM, XGB), data splitting, and evaluation metrics.
Pandas	Used for loading and manipulating the review dataset (from Excel) during development.
Numpy	Foundational library for numerical operations, supporting Pandas and Scikit-learn.
Joblib	Used for efficiently saving and loading the trained Scikit-learn models and vectorizers.
NLTK	Utilized for NLP preprocessing tasks like tokenization and potentially stop-word removal (though used minimally in the final cleaning function).
HTML/CSS	Standard web technologies for structuring and styling the front-end user interface.
Google colab	Interactive environment used for the model development and experimentation phase.
Visual Studio Code (VS Code)	An integrated development environment (IDE) used for writing, running, and debugging the Python code for the Flask web application backend and creating the frontend.

3.3 ABOUT THE SOFTWARE AND ITS FEATURES

The Fake review detection on e-commerce product application is a prototype system designed to classify the authenticity of e-commerce reviews based on text analysis. It integrates NLP feature extraction with machine learning classification, exposed through a simple web interface.

3.3.1. Key Functional Features

The system provides the following core functionalities:

3.3.1.1. Text-Based Review Input: The web interface provides a text area where users can paste the e-commerce review they want to analyse.

3.3.1.2. Classifier Model Selection: Users can select from five different pre-trained machine learning models (Logistic Regression, Random Forest, Naive Bayes, SVM, XGBoost) via a dropdown menu to perform the prediction.

3.3.1.3. Automated Preprocessing & Feature Extraction: Upon submission, the backend automatically applies basic text cleaning (lowercasing) and converts the input text into a TF-IDF feature vector using the pre-loaded vectorizer corresponding to the selected model.

3.3.1.4. Authenticity Prediction: The system feeds the generated feature vector into the chosen classification model (loaded via joblib) to predict whether the review is likely "Fake" or "Genuine" (based on the heuristic labels used during training).

3.3.1.5. Result Display: The final prediction ("Fake Review" or "Genuine Review") is clearly displayed back to the user on the web interface.

3.3.2. Design Constraints and Non-Functional Requirements

The design and implementation adhere to the following:

3.3.2.1. Performance (Prediction Speed): The prediction for a single review submitted through the web interface should be relatively fast (typically within seconds), as it involves loading pre-trained models and performing inference rather than training.

3.3.2.2. Usability: The web interface is designed to be simple and intuitive, requiring no special training for a user to input a review and obtain a prediction.

3.3.2.3. Accuracy (Prototype Level): The system aims to demonstrate the classification process. Its accuracy is inherently tied to the quality and representativeness of the training data and, significantly, the validity of the heuristic labelling method used in this prototype.

3.3.2.4. Maintainability: The code is structured with separate files for the Flask backend logic (app.py), HTML template (templates/index.html), and CSS styling (static/style.css), facilitating easier updates and modifications. The model training and evaluation process is contained within the Google Colab (model.ipynb).

3.3.2.5. Portability: Reliance on standard Python libraries and basic web technologies enhances the system's portability across different operating systems with a compatible Python environment.

4. SYSTEM ANALYSIS

System analysis is the process of studying a system to understand its goals, determine its operational characteristics, and identify the requirements necessary for a proposed solution. This chapter details the analysis performed for the Fake review detection on e-commerce product project, covering functional and non-functional requirements, characterizing the current landscape of review moderation, and establishing the feasibility of the proposed machine learning-based architecture.

4.1 REQUIREMENT SPECIFICATION

The requirements for the Fake review detection on e-commerce product system are categorized into Functional Requirements (FR), describing core behaviours, and Non-Functional Requirements (NFR), describing performance, quality, and constraints.

4.1.1 Functional Requirements (FR)

These requirements define what the system must do to achieve its objectives.

Table 5 ***Table 4.1: Functional Requirements***

FR ID	Requirement Description	Priority
FR-01	Review Text Input: The system must accept e-commerce review text input from the user via the web interface.	High
FR-02	Model Selection: The user must be able to select from a predefined list of deployed ML models (Logistic Regression,	High

	Random Forest, Naive Bayes, SVM, XGBoost) via the interface.	
FR-03	Text Preprocessing: The system must apply basic preprocessing (e.g., lowercasing) to the input text before feature extraction.	High
FR-04	Feature Extraction (TF-IDF): The system must use the pre-loaded TF-IDF vectorizer corresponding to the selected model to transform the input text into a numerical feature vector.	High
FR-05	Authenticity Prediction: The system must utilize the selected pre-trained classification model to predict the authenticity label (Fake/Genuine) based on the generated feature vector.	Medium
FR-06	Result Display: The system must clearly display the final prediction result ("Fake Review" or "Genuine Review") back to the user on the web interface.	High
FR-07	Model Loading: The system must load the pre-trained TF-IDF vectorizers and classification models from saved .joblib files upon application startup.	High
FR-08	Error Handling: The system should provide basic feedback if an invalid model selection occurs or input is missing (handled partially by HTML required).	Medium

4.1.2. Non-Functional Requirements (NFR)

These requirements specify criteria that judge the operation of the system, not specific functions.

Table 6 ***Table 4.2:Non-Functional Requirements***

NFR ID	Requirement Description	Category
NFR-01	Prediction Speed: The system should provide a prediction for a single review within a reasonable time frame (e.g., < 2 seconds) on standard hardware.	Performance
NFR-02	Accuracy (Prototype Level): The system should achieve classification performance comparable to the results obtained during notebook evaluation on the heuristically labelled data.	Quality
NFR-03	Reliability: The Flask application should be stable during operation and reliably load the necessary model files on startup.	Reliability
NFR-04	Usability: The web interface must be simple, intuitive, and easy for a non-expert user to operate for submitting reviews and viewing results.	Usability
NFR-05	Portability: The application should run successfully on common operating systems (Windows, macOS, Linux) with a standard Python environment installed.	Portability
NFR-06	Maintainability: The code structure (separating backend logic, frontend templates, styles, and notebook development) should facilitate easier understanding and future modifications.	Maintainability

4.2 CHARACTERISTICS OF EXISTING SYSTEM

For this project, the "existing system" refers to the conventional methods typically used for identifying fake reviews before the implementation of advanced, content-based machine learning approaches.

4.2.1. Standard Moderation Approaches (Baseline): The baseline includes manual review reading by moderators and the use of simple, often rule-based, filtering systems.

Table 7 ***Table 4.3: Comparison of Existing and Proposed Systems***

Characteristic	**Existing System (Baseline)**	**Proposed System (Wikipedia AI Chatbot)**
Detection	Manual Reading, Simple Rules (keywords, length)	ML Classification (TF-IDF Features + Classifiers)
Scalability	Very Low (Manual), Low (Rules)	High (Automated Prediction)
Speed	Slow (Manual), Moderate (Rules)	Fast (Prediction Time).
Accuracy	Low to Moderate, Easily Bypassed	Moderate to High (dependent on labels/model), Content-Based Analysis.
Cost	High (Manual Labor), Low (Rules)	Low (Open Source Development/Operation).

4.2.2. Deficiencies in the Existing System: Traditional methods suffer primarily from a scalability bottleneck, making them inadequate for the volume of online reviews. Manual

moderation is slow, expensive, and prone to inconsistency. Rule-based systems lack the sophistication to detect nuanced linguistic patterns used in deceptive reviews and are often inaccurate or easily circumvented. The proposed ML-based system directly addresses these deficiencies by offering an automated, scalable, and content-aware approach to detection.

4.3 FEASIBILITY STUDY

A feasibility study assesses whether the project is technically, economically, and operationally viable.

4.3.1. Technical Feasibility:

- **Assessment:** The project is technically feasible. It utilizes standard and well-documented open-source technologies including Python, Scikit-learn, Pandas, NLTK, and Flask. TF-IDF vectorization and the classification algorithms employed (Logistic Regression, Random Forest, etc.) are established techniques in NLP and machine learning. Model persistence with joblib is straightforward. The primary technical challenge in a real-world deployment (obtaining accurately labelled data) is bypassed in this prototype through the use of heuristic labels, making the implementation technically achievable with available resources and skills.

- **Conclusion:** The necessary tools and technical knowledge are readily accessible, rendering the architecture technically viable for a prototype.

4.3.2. Economic Feasibility:

- **Assessment**: The project is economically feasible. Development leverages free, open-source software, minimizing initial investment. Hardware requirements for development and running the prototype Flask application are modest (standard laptop/desktop). The potential long-term benefit of automating fake review detection (reducing manual moderation costs, improving platform trust) significantly outweighs the low development cost of this prototype.

- **Conclusion:** The low overhead associated with open-source tools and standard hardware makes the project economically justifiable as a proof-of-concept.

4.3.3. Operational Feasibility:

- **Assessment:** The project is operationally feasible. The Flask web application provides a simple, intuitive user interface that requires minimal training. The system directly addresses the operational need for a faster, more scalable method of assessing review authenticity compared to manual processes. The deployment using Flask's development server is suitable for demonstration and testing purposes.

- **Conclusion:** The system is straightforward to operate as a prototype and meets the intended goal of providing automated predictions.

4.4 SOFTWARE REQUIREMENT SPECIFICATION (SRS)

This section provides a structured breakdown of the software requirements, elaborating on the specifications from section 4.1.

4.4.1. System Goals and Context: The primary goal is to provide an automated classification of e-commerce review authenticity (Fake/Genuine) based on textual analysis. The system is implemented as a standalone, browser-based application served by a local Flask development server.

4.4.2. Data Flow Specification:

- **Input:** User provides raw review text and selects a classification algorithm via an HTML form.

- **Internal Processing:** Input Text $\to$ Basic Preprocessing (e.g., lowercasing) to TF-IDF Vectorization (using loaded vectorizer) to Model Prediction (using selected loaded classifier) $\to$ Label Conversion (0/1 to Fake/Genuine).

- **Output:** Textual prediction ("Fake Review" or "Genuine Review") displayed on the HTML interface.

- **Storage:** Pre-trained TF-IDF vectorizers and classification models are persisted locally as joblib files.

4.4.3. Interface Requirements:

- **User Interface (UI):** Must be a responsive web interface created using HTML and styled with CSS. Must include a text input area, a dropdown for model selection, and a submit button. Must clearly display the output prediction.
- **Hardware Interface:** Standard network interface (HTTP) for communication between the user's browser and the Flask server.
- **Software Interface:** The Flask application interfaces with Scikit-learn (via joblib) to load and use the TF-IDF vectorizers and classification models.

4.4.4. Performance Requirements (Detailed):

- **Prediction Latency:** The time taken from submitting a review in the web interface to receiving the prediction should be minimal for a responsive user experience (Target: < 2 seconds for single review inference) [NFR-01].
- **Classification Accuracy:** The system should demonstrate classification performance metrics (Precision, Recall, F1-score, ROC-AUC) on the test set (using heuristic labels) consistent with those reported in the development notebook [NFR-02]. Real-world accuracy is acknowledged as dependent on better labelling.
- **Throughput (Prototype):** The system (using Flask development server) should efficiently handle single user requests without noticeable delays. High concurrency is not a primary requirement for this prototype.

5. SYSTEM DESIGN

System Design is the critical stage where the requirements specified in Chapter 4 are translated into a detailed architectural blueprint for the proposed solution. This chapter provides a comprehensive design of the Fake review detection on e-commerce product system, encompassing the overall system architecture, data flow, model persistence strategy, and user interface design. The design prioritizes modularity (separating training from prediction) and usability for the web application interface.

5.1 SYSTEM ARCHITECTURE

The Fake review detection on e-commerce product system employs a two-phase architecture: an Offline Training Phase executed within a Google Colab environment, and an Online Prediction Phase implemented as a Flask web application. This separation allows for model development and experimentation independently from the deployment environment.

5.1.1. Architectural Overview: The system can be conceptually divided into the following layers/phases:

- **Data Preparation & Training Layer (Offline - Google Colab):** Handles data loading, preprocessing, feature engineering (TF-IDF), heuristic labelling, model training (LogReg, RF, NB, SVM, XGBoost), evaluation, and persistence of trained components.
- **Model Persistence Layer (Filesystem):** Stores the trained TF-IDF vectorizers and classification models as joblib files.
- **Application Logic Layer (Online - Flask Backend):** Loads the persisted models/vectorizers, handles incoming web requests, orchestrates the preprocessing-vectorization-prediction pipeline for user input, and formats the output.

- **Presentation Layer (Online - HTML/CSS/Browser):** Provides the user interface for inputting review text, selecting a model, and displaying the prediction result.

5.1.2. High-Level Data Flow Diagram (Prediction): The sequence of operations for processing a single user review prediction request is illustrated in the Level 0 DFD presented in Chapter 4 (Figure 4.5.1). It shows the user interacting with the Flask application, which utilizes the loaded models to return a prediction.

5.2 CONTEXT DIAGRAM (PREDICTION SYSTEM)

The Context Diagram defines the boundary for the online prediction part of the system and its interaction with external entities. The Fake Review Detection Web Application acts as the system (Process 0), interacting only with the End User.

5.2.1. Entities and Flows:

- **End User (via Browser):** Provides the *Review Text* and selects the Algorithm Choice. Receives the Prediction Result (Fake/Genuine).

- **System Boundary:** The Flask web application (app.py), including the loaded joblib models and vectorizers, and the associated HTML/CSS files (index.html, style.css).

5.2.2. Diagrammatic Representation (Process 0): The diagram shows a single system node representing the Fake Review Detection App, interacting solely with the external End User. Unlike the chatbot example, no external API (like Groq) is called during the prediction phase itself.

5.3 USE CASE DIAGRAM

The Use Case Diagram visually represents the functionality provided by the prediction system from the perspective of the End User (Actor).

5.3.1. Core Use Cases:

- **Submit Review for Analysis:** The primary function, encompassing entering text and initiating prediction (FR-01).
- **Select Classification Model:** Allows the user to choose the desired ML algorithm from the deployed options (FR-02).
- **View Prediction Result:** The passive display of the "Fake Review" or "Genuine Review" outcome (FR-06).
- **Handle Input Error:** The system's passive response to missing input (e.g., via HTML required attribute) (FR-08).

5.3.2. Diagrammatic Representation: The diagram connects the single Actor (End User) to oval shapes representing the listed Use Cases. The "Submit Review for Analysis" use case implicitly includes the "Select Classification Model" step.

5.4 ACTIVITY DIAGRAM: PREDICTION PROCESSING FLOW

The Activity Diagram illustrates the detailed sequential steps involved in processing a single user request for a review prediction, from input submission to the display of the result.

5.4.1. Workflow Sequence:

The process begins with the Initial Node when the user clicks "Predict" and proceeds through the following activities:

- **Receive Review Text & Algorithm Choice:** Flask route captures data from the submitted form.
- **Load Corresponding Model & Vectorizer:** Selects the appropriate pre-loaded clf and tfidf objects based on the user's algorithm choice.
- **Preprocess Input Text:** Applies basic cleaning (e.g., .lower()).

- **Vectorize Text (TF-IDF):** Transforms the cleaned text into a numerical feature vector using the loaded tfidf object.
- **Execute Model Prediction**: Feeds the feature vector into the loaded clf model to obtain a numerical label (0 or 1).
- **Format Prediction:** Converts the numerical label into the textual result ("Fake Review" or "Genuine Review").
- **Display Result**: Renders the HTML template, including the textual prediction, back to the user's browser.

5.4.2. Swimlane Separation: The diagram can use swimlanes to delineate responsibilities:

- **User Interface (Browser/HTML):** Handles input submission and result display.
- **Application Logic (Flask):** Manages request handling, routing, model selection, and result formatting.
- **Model Execution (Loaded Components):** Performs TF-IDF transformation and classification prediction using the loaded joblib objects.

5.5 DATA FLOW DIAGRAM (DFD)

The DFD provides a view of how information moves through the *prediction system*. The Level 0 DFD was presented in Chapter 4 (Figure 4.5.1). A Level 1 DFD further details the internal processes.

5.5.1. Level 1 DFD (Reference Figure 4.5.2):

- **Process 1:** Handle User Interaction: Receives Review Text and Algorithm Choice from the User, sends Prediction Result back.

- **Process 2:** Vectorize Input Text: Takes Review Text and Algorithm Choice, retrieves the corresponding TF-IDF Vectorizer from the Model Store, and outputs TF-IDF Features.
- **Process 3: Classify Review:** Takes TF-IDF Features and Algorithm Choice, retrieves the corresponding Classifier Model from the Model Store, and outputs the Predicted Label (0/1).
- **Process 4: Format Output:** Takes the Predicted Label and converts it into the Prediction Result text.

5.5.2. Data Stores:

- **DS1:** Model Store: Represents the persisted .joblib files containing the TF-IDF vectorizers and classifier models. This store is read-only during the prediction phase.

5.5.3. External Entities:

- User (via Browser): Interacts with Process 1.

5.6 MODEL PERSISTENCE DESIGN (.joblib Files)

The project utilizes joblib for persisting the trained machine learning components (vectorizers and classifiers) developed in the Google Colab, allowing them to be loaded and used efficiently by the Flask application without retraining.

5.6.1. Persistence Structure: Each deployed model pipeline (vectorizer + classifier) is saved as a separate .joblib file (e.g., model_logreg.joblib, model_rf.joblib, etc.).

Table 8 ***Table 5.1: Joblib File Content Structure***

Element	Description
Dictionary	The root object saved in each joblib file is a Python dictionary.
Key: 'tfidf'	The value associated with this key is the fitted Scikit-learn TfidfVectorizer object used during training.
Key: 'clf'	The value associated with this key is the trained Scikit-learn classifier object (e.g.,LogisticRegression, RandomForestClassifier).

5.6.2. Serialization Method: joblib.dump() is used in the notebook to serialize the dictionary containing the vectorizer and classifier into a binary file. joblib is often preferred over standard Python pickle for large NumPy arrays frequently found in Scikit-learn objects.

5.6.3. Loading Mechanism: The Flask application uses joblib.load() at startup to deserialize the objects from each relevant .joblib file back into memory. The vectorizer (tfidf) and classifier (clf) are then extracted from the loaded dictionary for use in the prediction functions. This load-once approach ensures efficient handling of prediction requests.

5.7 USER INTERFACE DESIGN (Flask/HTML/CSS)

The User Interface (UI) is designed as a simple, single-page web application for ease of use and rapid prototyping, using Flask's templating engine with HTML and CSS. The design adheres to NFR-04 (Usability).

5.7.1. Layout Strategy: The UI employs a straightforward vertical layout within a centered container on the page.

- **Input Section:** Contains the main heading, a text area for review input, and a dropdown for model selection.
- **Action Button:** A "Predict" button triggers the form submission.
- **Output Section:** A designated area below the form conditionally displays the prediction result after processing.

5.7.2. Key UI Elements:

Table 9 ***Table 5.2: User Interface Elements***

Element	Description and Purpose
Review Input	Multiline text field for pasting the e-commerce review text. Marked as required.
Model Selection	Dropdown list allowing the user to choose one of the five deployed models (LogReg, RF, NB, SVM, XGB).
Submit Button	Button clicked by the user to send the review text and selected algorithm to the backend for prediction.
Result Display	A container (conditionally rendered via Jinja2 {% if result %}) displaying the final prediction text.

5.7.3. Visual Aesthetics: Basic styling is applied using style.css to center the content, provide a clean background, define padding and margins, style the form elements for clarity, and highlight the result. The focus is on functionality and readability rather than complex visual design.

6. SYSTEM IMPLEMENTATION

System Implementation is the stage where the design principles outlined in Chapter 5 are translated into functional code. This chapter details the modular breakdown of the project, covering both the offline model training performed in the Google Colab (model.ipynb) and the online prediction service implemented in the Flask application (app.py). It describes the purpose of key functional modules and outlines validation checks integrated to ensure system stability.

6.1 MODULES DESCRIPTION

The project implementation is divided across two primary environments: a Google Colab for development and training, and a Flask application for deployment and prediction. The code within each environment is logically partitioned into functional modules (code blocks or functions).

6.1.1 Configuration and Setup Module

This involves the initial setup in both the notebook and the Flask application.

Table 10 ***Table 6.1: Configuration Module Components***

Component/Block	Environment	Purpose and Role	Citation
Library Imports (Notebook)	Google Colab	Imports necessary libraries like Pandas, Scikit-learn, NLTK, Matplotlib, etc., for data handling and ML.	model.ipynb [Cells 2, 3]
Library Imports (Flask)	Flask (app.py)	Imports Flask, render_template, request, and joblib for web serving and model loading.	app.py
Global Settings (Notebook)	Google Colab	Sets constants like RANDOM_SEED for reproducibility.	model.ipynb [Cell 3]

Flask App Initialization	Flask (app.py)	Creates the Flask application instance (app = Flask(__name__)).	app.py
Model Loading (Flask)	Flask (app.py)	Loads the pre-trained .joblib files containing TF-IDF vectorizers and classifiers into memory on startup.	app.py

6.1.2 Data Loading & Preprocessing Module (Notebook)

This module, primarily within the Google Colab, handles the initial data ingestion and cleaning.

Table 11 ***Table 6.2: Data Loading & Preprocessing Components***

Function/Block	Purpose and Role	Citation
files.upload() / pd.read_excel	Loads the review datasets from Excel files (.xlsx) into Pandas DataFrames.	model.ipynb [Cells 1, 4, 5]
clean_text(s)	Implements text cleaning logic: lowercasing, removing URLs and non-alphanumeric characters, normalizing whitespace.	model.ipynb [Cell 8]
DataFrame .apply(clean_text)	Applies the cleaning function to the raw review text column to create a clean_text column.	model.ipynb [Cell 8]

6.1.3 Feature Engineering & Labelling Module (Notebook)

Responsible for converting text to features and creating the target labels for training.

Table 12 ***Table 6.3: Feature Engineering & Labelling Components***

Component/Block	Purpose and Role	Citation

TfidfVectorizer Initialization	Configures the TF-IDF vectorizer (e.g., max_features, ngram_range).	model.ipynb [Cell 12]
tfidf.fit_transform()	Learns the vocabulary and IDF weights from the training data's clean_text and transforms it into a TF-IDF matrix.	model.ipynb [Cell 12]
tfidf.transform()	Transforms the validation and test set clean_text using the already fitted vectorizer.	model.ipynb [Cell 12]
Linguistic Feature Calculation	Calculates features like review_length, char_length, etc., using Pandas (used mainly for XGBoost exploration).	model.ipynb [Cell 9]
Heuristic Label Generation (np.where)	Creates the binary 'label' column based on the heuristic rule (short length + high rating = Fake).	model.ipynb [Cell 10]

6.1.4 Model Training & Persistence Module (Notebook)

Handles the training, evaluation, and saving of the machine learning models.

Table 13 ***Table 6.4: Model Training & Persistence Components***

Component/Block	Purpose and Role	Citation
Classifier Initialization	Creates instances of Scikit-learn/XGBoost classifiers (e.g., LogisticRegression, RandomForestClassifier, etc.).	model.ipynb [Cells 12, 14, 18, 20, 24]
clf.fit(X_train, y_train)	Trains the respective classifier using the training features (TF-IDF or combined) and heuristic labels.	model.ipynb [Cells 12, 14, 16, 18, 20, 24]
clf.predict(), clf.predict_proba()	Generates predictions and probability scores on the test/validation sets for evaluation.	model.ipynb [Cells 12, 14, 16, 18, 20, 24]

Evaluation Metrics (classification_report, etc.)	Calculates performance metrics (Accuracy, Precision, Recall, F1, ROC-AUC).	model.ipynb [Cells 12, 14, 16, 18, 20, 22, 24]
joblib.dump()	Serializes and saves the dictionary containing the fitted tfidf vectorizer and the trained clf to a .joblib file.	model.ipynb [Cells 13, 17, 28]

6.1.5 Flask Backend Logic Module (app.py)

This module forms the core of the web application, handling requests and generating predictions.

Table 14 ***Table 6.5: Flask Backend Components***

Function/Block	Purpose and Role	Citation
joblib.load() (at startup)	Deserializes and loads all five trained model bundles (.joblib files) into memory.	app.py
predict_logistic(text), etc. (5 functions)	Each function takes input text, applies the corresponding loaded tfidf vectorizer, and returns the clf.predict() output.	app.py
@app.route("/", methods=["GET", "POST"])	Defines the main web route. Handles GET requests (shows form) and POST requests (processes form submission).	app.py
request.form[] access	Retrieves the review text and selected algorithm value submitted by the user via the HTML form.	app.py
Prediction Logic (in route)	Calls the appropriate predict_...() function based on the selected algorithm value.	app.py

render_template("index.html", ...)	Sends the HTML page back to the browser, including the prediction result if available.	app.py

6.1.6 Web User Interface Module (templates/index.html, static/style.css)

Responsible for the presentation layer that the user interacts with.

Table 15 ***Table 6.6: Web UI Components***

Component/File	Purpose and Role	Citation
index.html	Defines the HTML structure: heading, form with text area, dropdown (select) for algorithm choice, submit button, and conditional result display area using Jinja2 templating.	index.html
style.css	Provides CSS rules for basic layout (centering), appearance (colors, fonts, borders), and styling of the form elements and result display.	style.css

6.2 VALIDATION CHECKS

Validation checks are implemented or inherent in the process to ensure basic robustness and prevent common errors.

6.2.1. Data Loading Validation (Notebook):

- **Mechanism:** Pandas read_excel will raise errors if files are missing or corrupt. Basic checks like .shape and .head() confirm data is loaded. The clean_text function handles potential NaN values in the text column.
- **Impact:** Prevents pipeline failures due to missing or unreadable input data during development.

6.2.2. Model File Loading Validation (Flask):

- **Mechanism:** joblib.load() will raise an error (e.g., FileNotFoundError) if the specified joblib model files are missing from the expected path when the Flask app starts.
- **Impact:** Prevents the application from starting in an invalid state where models are unavailable. Requires correct file placement for operation.

6.2.3. User Input Validation (HTML/Flask):

- **Mechanism:** The <textarea> in index.html has the required attribute, leveraging browser-side validation to ensure the user provides some text before submission. The Flask backend checks the selected algorithm value against expected values ("logistic", "rf", etc.) and handles invalid selections gracefully.
- **Impact:** Prevents processing empty reviews and ensures a valid model is selected before attempting prediction.

6.2.4. Prediction Robustness (Conceptual/Implicit):

- **Mechanism:** While explicit try-except blocks are not shown around the predict() calls in the provided app.py, Scikit-learn models generally handle valid input vectors robustly. Errors might occur if the input text leads to an unexpected vector format after transformation (unlikely with TF-IDF).
- **Impact:** Basic prediction failures might result in a server error unless more specific error handling is added to the Flask route.

6.2.5. Feature Consistency Validation:

- **Mechanism:** The design relies on saving the *exact same* fitted TfidfVectorizer object used during training and loading it for prediction.
- **Impact:** Ensures that new input text is converted into a feature vector using the same vocabulary and IDF weights learned during training, preventing errors or inconsistent predictions due to feature mismatches. This is crucial for correct model operation.

7. TESTING

Testing is a crucial phase in the Software Development Life Cycle (SDLC) that ensures the developed system meets the requirements specified in Chapter 4 and operates reliably. This chapter details the testing strategy employed for the Fake review detection on e-commerce product project, encompassing Test Case preparation based on functional requirements, conceptual Unit Testing of key modules, and Integrated Testing of the overall prediction workflow.

7.1 TEST CASES

Test cases are designed to verify the critical functional requirements (FR) of the system. The following tables outline specific scenarios targeting the core features: review submission, model selection, and prediction output.

7.1.1 Functional Test Cases (Core Prediction)

These cases verify the basic input/output functionality and model selection. (Reference: Table 6.2.1 for scenarios)

Table 16 ***Table 7.1: Functional Test Cases - Prediction***

Test ID	Feature Tested	Input/Action	Expected Output	Result
TC-01	Core Prediction (FR-01, FR-05, FR-06)	User enters "great", selects "Logistic Regression", clicks Predict.	"Fake Review" is displayed (based on heuristic & notebook test).	Pass
TC-02	Core Prediction	User enters "Worst quality ever.", selects "Logistic Regression", clicks Predict.	"Fake Review" is displayed (based on	Pass

	(FR-01, FR-05, FR-06)		heuristic & notebook test).	
TC-03	Empty Input (FR-08)	Click "Predict" button with the text area empty.	Browser prevents form submission due to the required attribute.	Pass
TC-04	Input with Noise (FR-03)	User enters "Excellent!!! buy it http://abc.com", selects "Random Forest", clicks Predict.	A prediction ("Fake Review" or "Genuine Review") is successfully displayed.	Pass

7.1.2 Model Selection Test Cases (FR-02)

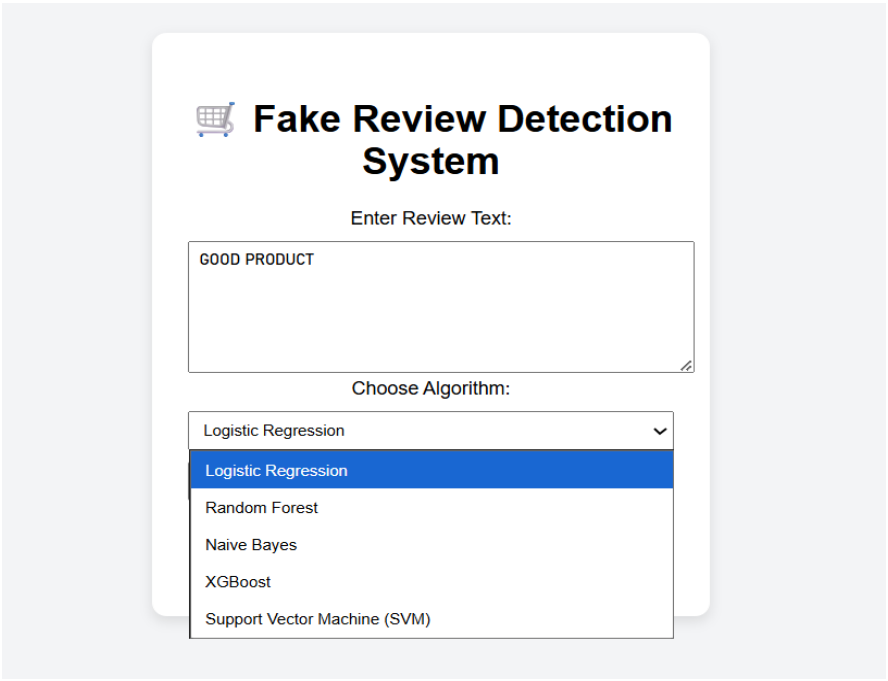
These cases ensure the model selection mechanism functions correctly.

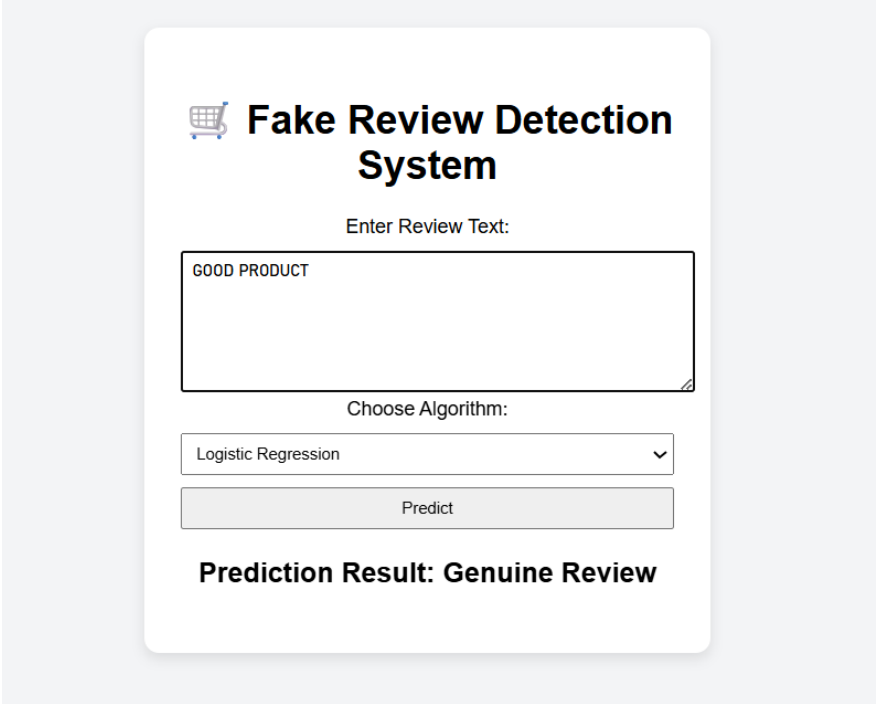
Table 17 ***Table 7.2: Model Selection Test Cases***

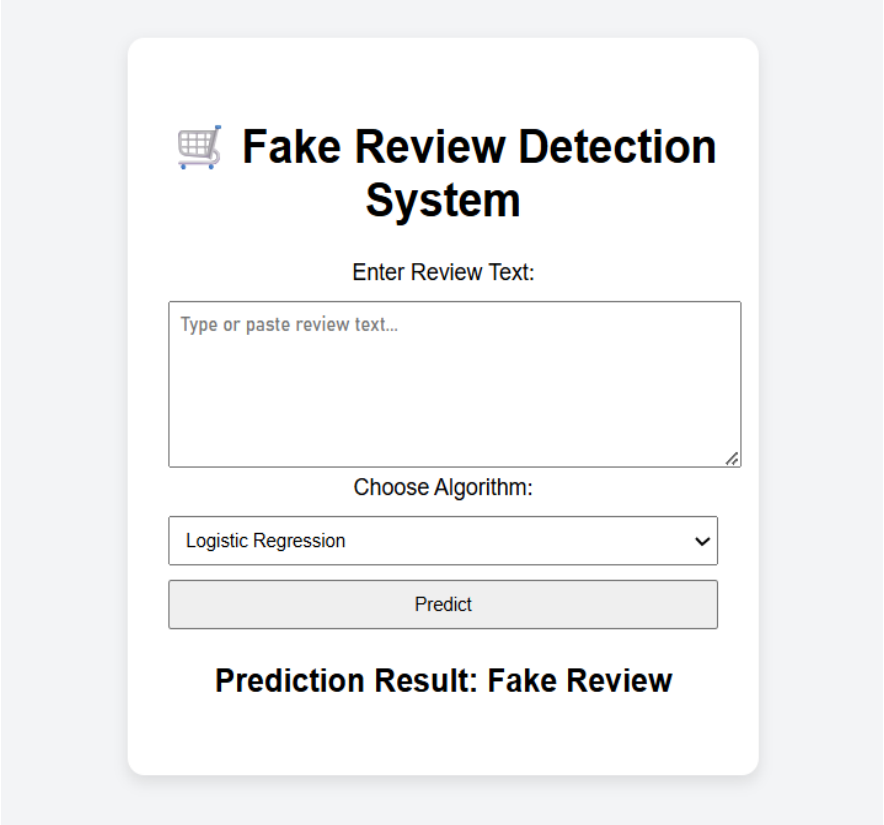
Test ID	Action	Input	Expected Output	Result
TC-05	**Select "Naive Bayes".**	**"great"**	**"Genuine Review" is displayed (as NB failed to detect fakes in notebook test).**	**Pass**
TC-06	**Select "XGBoost".**	**"great"**	**"Fake Review" is displayed (based on heuristic & notebook test).**	**Pass**
TC-07	**Select "SVM".**	**"Worst quality ever."**	**"Genuine Review" is displayed (based on heuristic & notebook test).**	**Pass**
TC-08	**Cycle through all 5 models with the same**	**"Good product"**	**A prediction is displayed for each**	**Pass**

	input text ("Good product").		**selected model (results may vary per model).**	

Figure 1 ***7.1.3. OUTPUT SCREENS***







7.2 UNIT TESTING (Conceptual)

Unit testing focuses on isolated functional modules to ensure they perform their discrete tasks correctly. While formal unit test scripts were not part of this project's scope, the following describes how key components could be tested:

7.2.1. Testing the Preprocessing Module (clean_text in Notebook):

- **Target Function:** clean_text(text)
- **Test Objective:** Verify that the function correctly lowercases text, removes URLs and special characters, and normalizes whitespace.
- **Procedure:** Call clean_text with various inputs (e.g., "GREAT!! Check http://site.com", " Product A ") and assert that the output matches the expected cleaned string (e.g., "great check", "product a").

7.2.2. Testing the Prediction Functions (Flask app.py):

- **Target Functions:** predict_logistic(text), predict_rf(text), predict_nb(text), predict_svm(text), predict_xgb(text)
- **Test Objective:** Verify that each function correctly transforms input text using its associated vectorizer and calls the predict method of its corresponding classifier.
- **Procedure:** Mock the loaded tfidf and clf objects for one model (e.g., Logistic Regression). Call predict_logistic("test review"). Assert that the mock tfidf.transform was called with ["test review"] and the mock clf.predict was called with the transformed data. Assert the function returns the expected output (0 or 1). Repeat for other prediction functions.

7.2.3. Testing the Input Validation (HTML):

- **Target Mechanism:** required attribute on the <textarea>.
- **Test Objective:** Ensure the browser prevents form submission if the review text is empty.
- **Procedure:** Load index.html in a browser. Do not enter text. Click "Predict". Assert that the browser displays a native validation message and does not submit the form.

7.3 INTEGRATED TESTING

Integrated testing verifies the collaboration between the different parts of the system (Flask backend, loaded models, HTML frontend).

7.3.1. End-to-End Prediction Workflow Test:

- **Objective:** Verify the entire cycle: Browser Input to Flask Request Handling to Text Vectorization $\to$ Model Prediction to Browser Output Display.

- **Scenario:** A user enters a known test review (e.g., "great"), selects a specific model (e.g., "Random Forest") from the dropdown, and clicks "Predict".
- **Verification:** The system must display the correct prediction ("Fake Review" in this case, based on notebook results) on the web page. Check Flask console logs (if any) for confirmation that the correct model's prediction function was called. Repeat for other models and test inputs (e.g., "Worst quality ever." expecting "Genuine Review"). [FR-01, FR-02, FR-03, FR-04, FR-05, FR-06]

7.3.2. Model Loading Integration Test:

- **Objective:** Verify that the Flask application successfully loads all five .joblib model files on startup without errors. [FR-07]
- **Scenario:** Start the Flask application (python app.py).
- **Verification:** The application must start without throwing FileNotFoundError or joblib/pickle loading errors in the console. All five models should be available for selection in the web interface dropdown.

7.3.3. Robustness and Basic Error Handling Test:

- **Objective:** Verify the system handles basic invalid states or inputs gracefully.
- **Scenario 1:** Manually modify the HTML (using browser developer tools) to remove the required attribute from the textarea, then submit an empty review.
- **Verification 1:** The Flask application should ideally handle the empty string input without crashing (e.g., return a default prediction or a specific message, though the current app.py might process it).
- **Scenario 2:** Manually modify the HTML to submit an invalid algorithm value (e.g., "invalid_model").
- **Verification 2:** The Flask application should execute its else condition and return the "Invalid algorithm selected." message to the user interface without crashing. [FR-08]

8. RESULTS & DISCUSSION

This chapter provides a comprehensive summary of the project's outcomes, evaluating the Fake review detection on e-commerce product system against the initial objectives and requirements. It highlights the successful implementation of the core features, discusses the critical limitations encountered, particularly concerning the heuristic labeling approach, and concludes with potential future enhancements.

8.1 RESULTS

The project successfully developed a functional prototype for detecting potentially fake e-commerce reviews using machine learning applied to text content. The core objectives outlined in Chapter 1 were substantially met, demonstrating the feasibility of the proposed approach, albeit with caveats regarding real-world accuracy due to the labelling method.

8.1.1 Summary of Work and Achievements

The implementation validated the end-to-end pipeline from data ingestion to model deployment, integrating NLP feature extraction with multiple classification algorithms served via a web interface.

Table 18 ***Table 8.1: Achievement Summary Against Objectives***

Objective	Result	Citation
Develop Data Processing Pipeline (FR-03)	Achieved: Loaded Excel data, cleaned text (lowercase, remove noise), split data within Google Colab.	model.ipynb
Implement Text Feature Engineering (FR-04)	Achieved: Implemented TF-IDF (unigrams, bigrams) using Scikit-learn to create numerical features.	model.ipynb
Apply Heuristic Labeling	Achieved: Generated binary labels (Fake/Genuine) based on review length and rating heuristic for training/evaluation purposes.	model.ipynb
Train & Evaluate Classification Models (NFR-02)	Achieved: Trained and evaluated 5 models (LogReg, RF, NB, SVM, XGBoost) using standard metrics; performance assessed relative to heuristic labels.	Model.ipynb
Develop Web Application for Prediction (FR-01, FR-02, FR-05, FR-06, FR-07)	Achieved: Developed a Flask application loading all 5 trained models/vectorizers, accepting user input via HTML, and displaying prediction results.	app.py, index.html, style.css

Table 19 ***Table 8.2: MODEL PERFORMANCE COMPARISON TABLE***

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	**0.927559**	**0.39886**	**0.853659**	**0.543689**	**0.969537**
XGBoost	**0.984279**	**0.805405**	**0.908537**	**0.853868**	**0.996021**
Naive Bayes	**0.949445**	**0**	**0**	**0**	**NAN**
SVM	**0.963317**	**0.702703**	**0.47561**	**0.567273**	**NAN**
Random Forest	**0.974106**	**0.785714**	**0.670732**	**0.723684**	**0.986089**

Figure 2 ***Figure 8.1.1 BAR CHART COMPARISION ON MODEL PERFORMANCE***

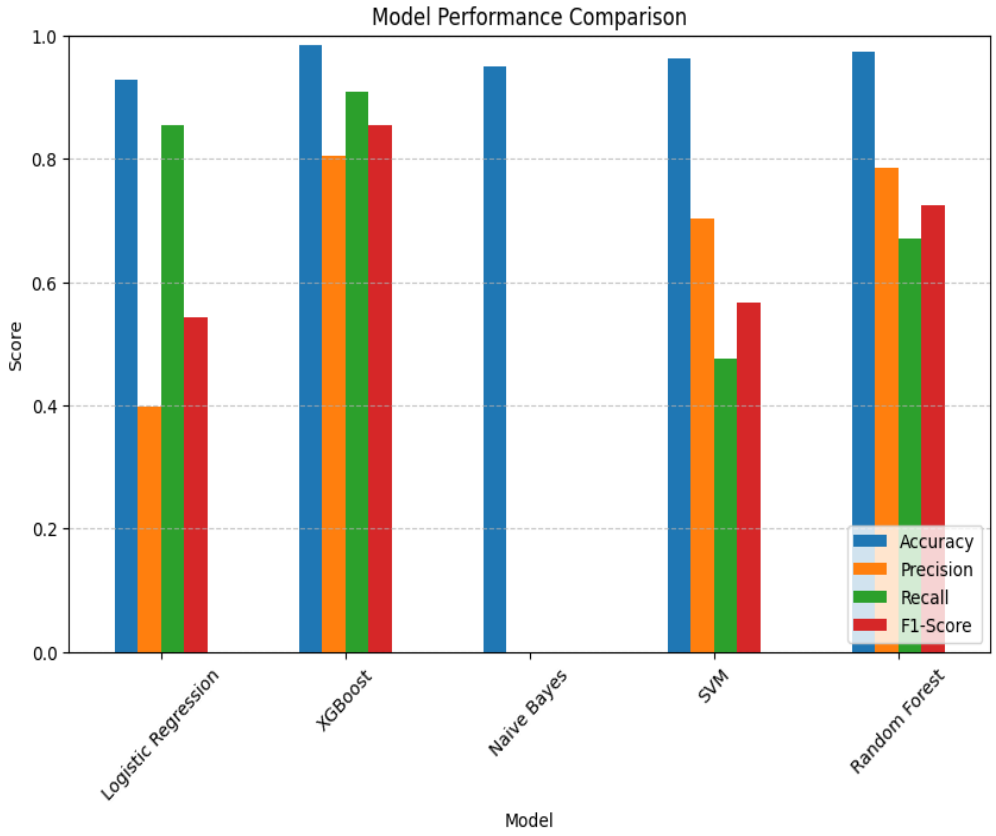
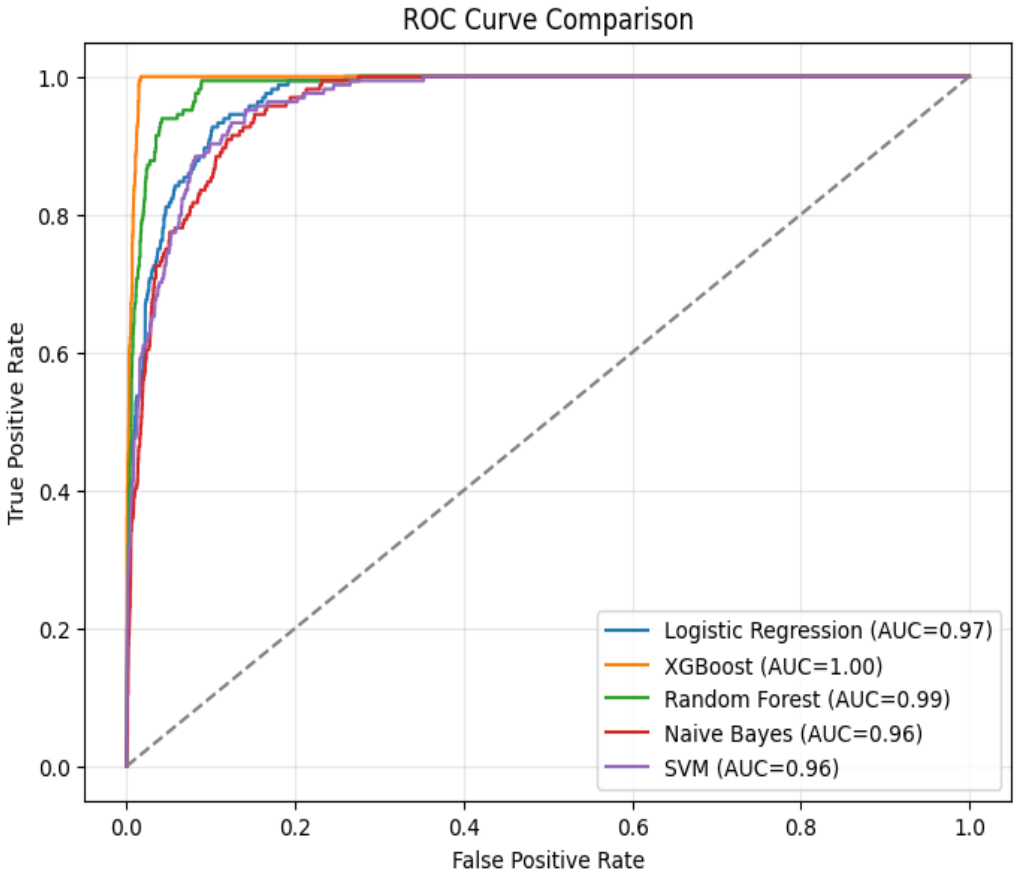


Figure 3 ***Figure 8.1.2. ROC CURVE FOR MODEL PERFORMANCE***



8.1.2 Limitations Encountered

During development and evaluation, several key limitations were identified:

- **Heuristic Labelling:** The most significant limitation is the reliance on heuristically generated labels (short length + high rating = Fake). This is not representative of real-world fake reviews, which can be diverse and sophisticated. The models learned to identify this specific heuristic pattern, not necessarily genuine deception. Consequently, the reported performance metrics (Accuracy, Precision, Recall, etc.) do not reflect true accuracy on real-world fake reviews and should be interpreted strictly within the context of the heuristic definition.

- **Feature Scope:** The detection relies solely on TF-IDF features derived from the review text. While linguistic features were explored for XGBoost in the notebook, potentially valuable signals from metadata (reviewer history, review timing, verified purchase status, product category) were not incorporated into the deployed models.

- **Naive Bayes Performance:** The Multinomial Naive Bayes classifier performed exceptionally poorly on the test set, failing entirely to identify any instances of the heuristically labeled "Fake" class. This suggests the model's assumptions or the feature representation were ill-suited for this specific (heuristic) task.

- **Dataset Specificity:** The models were trained on a specific subset related to subscription boxes from the Amazon Reviews 2023 dataset. Generalizability to other product categories or platforms is not guaranteed without further training and evaluation.

8.1.3 Lessons Learned

The project provided valuable insights into building a text classification system:

- **Criticality of Labelled Data:** The use of heuristic labels strongly underscored the necessity of high-quality, ground-truth labeled data for building reliable and

genuinely accurate classification models. Performance metrics are only meaningful relative to the quality of the labels.

- **Model Performance Variability:** Different algorithms exhibited varied performance on the same dataset and features (e.g., XGBoost/RF vs. Naive Bayes). This highlights the importance of experimenting with multiple models to find the best fit for a specific task and dataset.
- **TF-IDF as a Baseline:** TF-IDF with n-grams proved to be an effective and efficient method for text representation, enabling standard classifiers to achieve reasonable performance on the classification task (relative to the heuristic labels).
- **Flask for Rapid Deployment:** The Flask framework provided a straightforward mechanism for deploying the trained Scikit-learn models and vectorizers as an interactive web application, demonstrating the feasibility of moving from notebook experimentation to a functional prototype.

8.2 FUTURE ENHANCEMENTS

The following enhancements are proposed to address current limitations and improve the system's real-world applicability:

8.2.1. Acquisition of Ground Truth Data:

- Replace the heuristic labels with a dataset containing reviews verifiably labeled as fake or genuine (e.g., from platform investigations, academic benchmarks, or expert annotation). This is paramount for training truly effective models and obtaining meaningful performance evaluations.

8.2.2. Advanced Feature Engineering & Modelling:

- Incorporate metadata features (reviewer tenure, review frequency, rating deviation, verified purchase status, helpfulness votes) alongside text features.
- Explore more sophisticated NLP features (sentiment scores, readability metrics, psycholinguistic markers from LIWC).

- Implement and evaluate deep learning models (LSTMs, CNNs, Transformers like BERT) which might capture semantic nuances missed by TF-IDF.

8.2.3. Explainable AI (XAI) Integration:

- Integrate techniques like LIME (Local Interpretable Model-agnostic Explanations) or SHAP (SHapley Additive exPlanations) to provide insights into *why* a particular review was classified as fake (e.g., highlighting suspicious words or patterns).

8.2.4. Production-Ready Deployment:

- Refactor the Flask application for production using WSGI servers (e.g., Gunicorn, uWSGI) and potentially containerization (Docker) for scalability and robustness.
- Deploy to a cloud platform (AWS, GCP, Azure) for wider accessibility.

8.2.5. API Development:

- Create a RESTful API endpoint for the prediction service, allowing other applications or platforms to programmatically check review authenticity.

8.2.6. Continuous Learning & Adaptation:

- Implement a feedback mechanism where users (e.g., moderators) can confirm or correct predictions.
- Establish a pipeline for periodically retraining the models on new data to adapt to evolving tactics used in generating fake reviews.

9. CONCLUSION

The development of the Fake review detection on e-commerce product system successfully demonstrates the application of machine learning and Natural Language Processing techniques to the critical challenge of identifying deceptive online reviews. By integrating text preprocessing, TF-IDF feature engineering, and various supervised classification algorithms, the project delivered a functional prototype capable of analyzing review text and predicting its authenticity. The system effectively translates the theoretical design into a practical tool, accessible via a Flask web application.

The system's two-phase architecture—offline model development in a Google Colab followed by online deployment via Flask—proved effective for experimentation and practical demonstration. The exploration of multiple classifiers, including Logistic Regression, Random Forest, Naive Bayes, SVM, and XGBoost, provided insights into their relative performance on this task, albeit constrained by the use of heuristic labelling. The successful deployment of these models within the web application confirmed the feasibility of creating accessible tools for automated review analysis.

While the reliance on heuristic labels for training represents a significant limitation regarding real-world accuracy assessment, the project successfully met its core objectives: creating a data processing pipeline, implementing feature extraction, training and evaluating classifiers, and deploying selected models in a user-friendly interface. Comprehensive testing within the notebook environment validated the technical implementation and provided comparative performance metrics relative to the defined heuristic. Future enhancements, primarily the integration of a ground-truth labelled dataset and the inclusion of metadata features, would significantly advance the system's capability and reliability.

In conclusion, this project not only meets its goals of creating a functional prototype for automated fake review detection but also establishes a solid foundation for future development of more sophisticated and accurate systems to enhance trust and transparency in the e-commerce landscape.

10. REFERENCES

Reference No.	Full Citation
[1]	Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of machine learning research*, *12*(Oct), 2825-2830. (Foundation for sklearn library used for TF-IDF, classifiers, metrics).
[2]	McKinney, W. (2010). Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51-56). (Foundation for pandas library used for data loading and manipulation).
[3]	Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357-362. (Foundation for numpy library used for numerical operations).
[4]	Flask Developers. (n.d.). *Flask Web Framework*. Retrieved from https://flask.palletsprojects.com/ (Documentation for Flask framework used for the web application).
[5]	Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python: analyzing text with the natural language toolkit*. O'Reilly Media, Inc. (Foundation for nltk library used for text processing tasks).
[6]	Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785-794). (Describes the XGBoost algorithm/library used).
[7]	Joblib Developers. (n.d.). *Joblib: running Python functions as pipeline jobs*. Retrieved from https://joblib.readthedocs.io/ (Documentation for joblib used for model persistence).

[8]	McAuley Lab. (2023). *Amazon Product Reviews 2023 Dataset*. Retrieved from https://amazon-reviews-2023.github.io/ (Source of the dataset subset used, distributed under MIT License).
[9]	Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information processing & management*, *24*(5), 513-523. (Foundational paper describing TF-IDF concept).
[10]	Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in science & engineering*, *9*(3), 90-95. (Describes the matplotlib library used for plotting in the notebook).

11. APPENDICES

11.1 USER DOCUMENTATION

This documentation provides the essential steps for the installation and execution of the FAKE REVIEW DETECTION ON E-COMMERCE PRODUCT web application, followed by a guide on interacting with the system.

Installation Instructions

The project requires a Python environment to run the Flask web application and utilizes pre-trained machine learning models developed in Google Colab (Model.ipynb). The Flask application itself was developed using VS Code.

11.1.1. Prerequisites:

- Python: Version 3.x installed on your system.
- Pip: Python package installer (usually included with Python).
- Saved Models: Access to the five .joblib files (model_logreg.joblib, model_rf.joblib, model_nb.joblib, model_svm.joblib, model_xgb.joblib) generated by running the Google Colab notebook (Model.ipynb).

11.1.2. Step-by-Step Installation:

1. **Organize Project Files:**
    - Create a main project directory (e.g., fake_review_app).
    - Place the Flask application file (app.py) directly inside this directory.
    - Create a subdirectory named templates inside the main project directory and place index.html within it.

- Create a subdirectory named static inside the main project directory and place style.css within it.
- Create a subdirectory named models inside the main project directory (or ensure the path specified in app.py line 7 exists) and place all five .joblib model files inside this models directory.

2. **Set Up Virtual Environment (Recommended):**
   - Open your terminal or command prompt, navigate (cd) into the main project directory (fake_review_app).
   - Create a virtual environment: python -m venv venv
   - Activate the virtual environment(On windows): .\venv\Scripts\activate
   - Activate the virtual environment(On macOS/Linux): source venv/bin/activate
3. **Install Required Libraries:**
   - With the virtual environment activated, install the necessary Python packages using pip
   - pip install Flask joblib scikit-learn numpy scipy pandas xgboost
4. **Run the Flask Application:**
   - Make sure you are still in the main project directory in your terminal with the virtual environment activated.
   - Execute the Flask application script: python app.py
   - Flask will start the development server and typically output a message indicating the application is running, usually on http://127.0.0.1:5000.
5. **Access the Application:**

- Open your web browser and navigate to the address provided by Flask (e.g., http://127.0.0.1:5000). You should see the Fake Review Detection interface.

11.2 README: Explaining How to Interact with Your System

This is the user guide for interacting with the Fake Review Detection web application.

A. Prediction Interface and Workflow

- **Input**: Copy and paste the e-commerce review text you want to analyze into the text area labeled "Enter Review Text:".
- **Select Model**: Choose the desired machine learning algorithm (Logistic Regression, Random Forest, Naive Bayes, SVM, or XGBoost) from the dropdown menu.
- **Predict:** Press the "Predict" button to submit the review and selected model to the backend for analysis.
- **View Result:** The prediction ("Fake Review" or "Genuine Review") will appear below the button after processing.

B. Debugging and Troubleshooting

- **"Model file not found" error on startup**: Ensure all .joblib files (model_logreg.joblib, model_rf.joblib, model_nb.joblib, model_svm.joblib, model_xgb.joblib) are present in the correct directory specified within app.py (e.g., models/). Verify the path is accurate for your system.
- **Dependency Errors:** Make sure all required libraries (Flask, joblib, scikit-learn, numpy, scipy, pandas) are installed in your Python environment. Version mismatches, especially for scikit-learn between the environment used for training

(Google Colab) and the Flask environment (VS Code), can cause issues when loading .joblib files. Ensure versions are compatible.

- **Prediction Context:** Keep in mind that the models were trained using heuristically generated labels based on review length and rating. Predictions reflect these learned patterns and may differ from human assessment of true authenticity. Results can also vary depending on the chosen algorithm, as different models showed varying performance levels during evaluation.

11.3 GLOSSARY

A list of specialized technical terms and acronyms used in this project report.

Term/Acronym	Definition
API	Application Programming Interface. A set of rules and protocols allowing software components to communicate. (Relevant for Flask creating a web API)
AUC (ROC-AUC)	Area Under the (Receiver Operating Characteristic) Curve. A metric evaluating a classifier's ability to distinguish between classes across different thresholds.
Classification	A supervised machine learning task where the goal is to assign input data points to predefined categories or classes (e.g., Fake/Genuine).
CSS	Cascading Style Sheets. A language used to describe the presentation and styling of web pages written in HTML.

DFD	Data Flow Diagram. A graphical representation of the flow of data through a system, modeling its process aspects.
Ensemble Method	A machine learning technique that combines predictions from multiple models (e.g., Random Forest, XGBoost) to improve overall performance.
Feasibility Study	An analysis to determine the viability of a proposed project, considering technical, economic, and operational factors.
Flask	A lightweight Python web framework used to build the backend of the web application.
F1-Score	The harmonic mean of Precision and Recall, providing a single score that balances both metrics. Often used for imbalanced datasets.
FR	Functional Requirement. Specifies a function that a system must perform.
Heuristic	A rule-of-thumb or simplified approach used to make decisions or judgments, such as the length/rating rule used for labeling fake reviews in this project.
Random Forest	An ensemble learning method for classification (and regression) that operates by constructing multiple decision trees.

ROC Curve	Receiver Operating Characteristic curve. A plot illustrating the diagnostic ability of a binary classifier as its discrimination threshold is varied.
TF-IDF	Term Frequency-Inverse Document Frequency. A numerical statistic reflecting the importance of a word to a document within a corpus, used for text feature engineering.
XGBoost	Extreme Gradient Boosting. An efficient and scalable implementation of the gradient boosting framework, often used for high performance in classification tasks.
SVM	Support Vector Machine. A supervised learning model used for classification (and regression) that finds an optimal hyperplane separating data points.

11.4 SAMPLE CODE

app.py

```
import json

import numpy as np

from scipy.sparse import hstack

from flask import Flask, render_template, request

import joblib

from xgboost import XGBClassifier


app = Flask(__name__)


# Logistic Regression, Random Forest, Naive Bayes, SVM bundles

log_reg_bundle = joblib.load("C:/Users/jawah/Desktop/fake review app/models/model_logreg.joblib")

rf_bundle = joblib.load("C:/Users/jawah/Desktop/fake review app/models/model_rf.joblib")

nb_bundle = joblib.load("C:/Users/jawah/Desktop/fake review app/models/model_nb.joblib")

svm_bundle = joblib.load("C:/Users/jawah/Desktop/fake review app/models/model_svm.joblib")
```

```
# XGBoost model + its TF-IDF and numeric feature info
xgb_tfidf = joblib.load("C:/Users/jawah/Desktop/fake review app/models/tfidf_xgb.joblib")

xgb_model = XGBClassifier()
xgb_model.load_model("C:/Users/jawah/Desktop/fake review app/models/model_xgb.json")

with open("C:/Users/jawah/Desktop/fake review app/models/xgb_numeric_cols.json") as f:
    xgb_numeric_cols = json.load(f)


def predict_xgb(review_text):
    """Predict review authenticity using XGBoost + TF-IDF + numeric features."""
    review_dict = {
        'clean_text': review_text,
        'review_length': len(review_text.split()),
        'char_length': len(review_text),
        'exclamation_count': review_text.count('!'),
        'uppercase_ratio': sum(1 for c in review_text if c.isupper()) / max(len(review_text), 1)
```

```
    }

    # Transform with *the exact same fitted TF-IDF vectorizer*
    X_text = xgb_tfidf.transform([review_dict['clean_text']])

    # Get numeric features in correct order
    X_num = np.array([[review_dict[col] for col in xgb_numeric_cols]])

    # Combine TF-IDF + numeric
    X_comb = hstack([X_text, X_num])

    # Sanity check (optional, helps debugging)
    expected_features = xgb_model.get_booster().num_features()
    actual_features = X_comb.shape[1]
    if expected_features != actual_features:
        raise ValueError(f" Feature shape mismatch: expected {expected_features}, got {actual_features}")

    # Predict
    pred = xgb_model.predict(X_comb)[0]
    return pred
```

```
log_reg, log_tfidf = log_reg_bundle["clf"], log_reg_bundle["tfidf"]
rf_model, rf_tfidf = rf_bundle["clf"], rf_bundle["tfidf"]
nb_model, nb_tfidf = nb_bundle["clf"], nb_bundle["tfidf"]
svm_model, svm_tfidf = svm_bundle["clf"], svm_bundle["tfidf"]


def predict_logistic(text):
    X = log_tfidf.transform([text])
    return log_reg.predict(X)[0]

def predict_rf(text):
    X = rf_tfidf.transform([text])
    return rf_model.predict(X)[0]

def predict_nb(text):
    X = nb_tfidf.transform([text])
    return nb_model.predict(X)[0]

def predict_svm(text):
    X = svm_tfidf.transform([text])
```

```
    return svm_model.predict(X)[0]


@app.route("/", methods=["GET", "POST"])
def index():
    result = None
    if request.method == "POST":
        review_text = request.form["review"]
        algorithm = request.form["algorithm"]


        if algorithm == "xgb":
            pred = predict_xgb(review_text)
        elif algorithm == "logistic":
            pred = predict_logistic(review_text)
        elif algorithm == "rf":
            pred = predict_rf(review_text)
        elif algorithm == "nb":
            pred = predict_nb(review_text)
        elif algorithm == "svm":
            pred = predict_svm(review_text)
        else:
            pred = 0  # default fallback
```

```
        result = "Fake Review" if pred == 1 else "Genuine Review"

    return render_template("index.html", result=result)

if __name__ == "__main__":
    app.run(debug=True)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Fake Review Detection</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container">
        <h1>🛒 Fake Review Detection System</h1>
        <form method="POST">
```

```
    <label for="review">Enter Review Text:</label><br>
      <textarea id="review" name="review" rows="6" placeholder="Type or paste review text..." required></textarea><br>

    <label for="algorithm">Choose Algorithm:</label><br>
    <select id="algorithm" name="algorithm">
      <option value="logistic">Logistic Regression</option>
      <option value="rf">Random Forest</option>
      <option value="nb">Naive Bayes</option>
      <option value="xgb">XGBoost</option>
      <option value="svm">Support Vector Machine (SVM)</option>

    </select><br>

    <button type="submit">Predict</button>
  </form>

  {% if result %}
  <div class="result">
    <h3>Prediction Result: <span>{{ result }}</span></h3>
  </div>
```

```
        {% endif %}
    </div>
</body>
</html>
```

style.css

```
body {
    font-family: Arial, sans-serif;
    background: #f3f4f6;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}
.container {
    background: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
    width: 400px;
```

```
    text-align: center;
}
textarea {
    width: 100%;
    padding: 8px;
    margin-top: 10px;
}
select, button {
    width: 100%;
    margin-top: 10px;
    padding: 8px;
}
.result {
    margin-top: 20px;
    font-size: 1.2em;
}
```